CSE 213 — Software Engineering

Creational Design Patterns — Factory Method, Abstract Factory, Singleton, Builder

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Anindya Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — creational-pattern questions by year
Lesson 5 — UML Recap and Factory Method — sl. 2–16; travel packages, parser by extension
Lesson 6 — Abstract Factory — sl. 17–35; pizza regions, set menus, Uber/Pathao
Lesson 7 — Singleton — sl. 36–42; eager/lazy, serial id generator, limited instances
Lesson 8 — Builder and Choosing a Creational Pattern — sl. 43–53; HTML page, forms, set meals, burger shop, comparison
Quick Notes — Creational Patterns — revision + key structures

> **NOTE** Slide numbers refer to the PDF page numbers of the deck named under each lesson title. TF page numbers refer to 'CSE 213 All Past Papers.pdf' (70 pages); the newest paper (22 batch, labelled 2024-25 here) is a separate file, cited as '22 batch p.N'. Old course code: CSE 307. Slide numbers refer to 'Lec 3 - Creational Design Patterns.pdf' (53 slides). All Java in this file was compiled and run with javac (sources in _build/java/A2).

Exam Map — What the TF Papers Ask

Session (paper)	Questions in this note file
2024-25 (CSE 213)	Q1(a) Travel packages (Adventure/Regular): centralised creation, client never names concrete class, add types without modifying — pattern + Java (15) [L5]
2023-24 (CSE 213)	Q5(a) Pizza stores Italy / America with regional styles — pattern + code (15) [L6]
2022-23 (CSE 307)	Q3(a) HTMLPageDirector / HTMLPageBuilder — class diagram + code (15) [L8]
2021-22 (CSE 307)	Q1(a) Differentiate use cases of Factory Method, Abstract Factory, Builder and Bridge (16) [L8] · Q3(a) Restaurant set menus (Veggie Harvest, Chicken Delight, Beef Fiesta, Sea Shanty) from three kitchens — pattern + UML + code (20) [L6]
2020-21 (CSE 307)	Q1(a) Burger shop: pizzas + extra cheese + drinks + appetizers + meals — construct the four orders with three different patterns, with prices (28) [L8] · Q1(b) Advantages and disadvantages of the patterns used in 1(a) (7) [L8]
2019-20 (CSE 307)	Q1(a) Serial unique identifiers (userID, district, qualification, TxID) starting from 1 — suitable pattern (15) [L7]
2018-19 (CSE 307)	Q3(b) Forms built per user type (Builder) — class diagram + code (20) [L8]
2017-18 (CSE 307)	Q6(a) Lalbagh set meals (rice + curries / bread + kebabs, optional drinks, bill) — class diagram + code (22) [L8] · Q7(b) Uber / Pathao car and moto fare formulas — class diagram + code (21) [L6]
2016-17 (CSE 307)	Q5(a) Eager vs lazy singleton; when lazy is preferred (15) [L7] · Q5(c) Choose parser by file extension (.c, .cpp, .java, .html) — pattern + Java (10) [L5]
2014-15 (CSE 307)	Q6(b) Pattern that reduces dependence on hardware/software platforms (12) [L6]
2012-13 (CSE 307)	Q2(a) Lazy vs eager singleton; structure (10) [L7]
2011-12 (CSE 307)	Q1(b) Base class with Derived1 (max 5 instances) and Derived2 (max 10) — Singleton-style Java (18⅔) [L7]
2010-11 (CSE 307)	Q3(d) Does XMLReaderFactory implement the Factory pattern? (5) [L5] · Q4(a) Which singleton implementation (eager / holder) is valid? (5) [L7]

EXAM FOCUS The highest-scoring part of Anindya Sir's section: a 15–28-mark 'identify the pattern, draw the class diagram, write the code' question appears in every paper since 2017-18 (2024-25 Factory Method, 2023-24 Abstract Factory, 2022-23 Builder, 2021-22 Abstract Factory + comparison, 2020-21 three-pattern burger shop, 2019-20 Singleton, 2018-19 Builder, 2017-18 Builder + Abstract Factory). Learn the four structures by heart, then the cue words in the Lesson 8 decision table.

Lesson 5 — UML Recap and Factory Method

Slides 2–16 (Lec 3 - Creational Design Patterns)

1. UML relationships used in pattern diagrams (sl. 2)

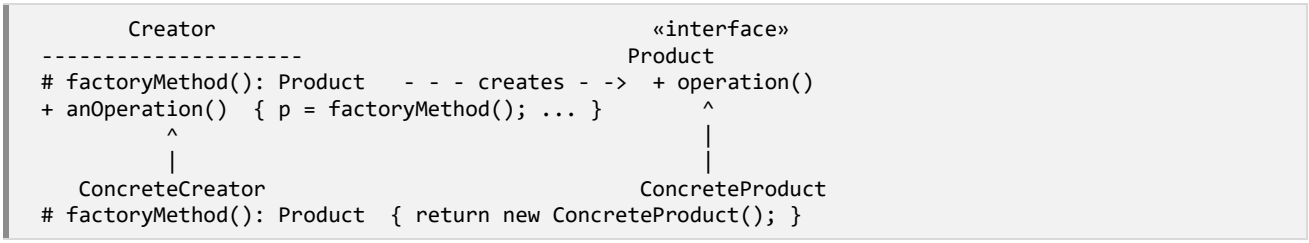
Relationship	Arrow	Reading	Slide example
Generalization	solid line, hollow triangle → parent	is-a / implements	Rectangle → Shape
Dependency	dashed line, open arrow	uses temporarily	Lottery ⇢ Random
Aggregation	hollow diamond at the whole	is part of (part can live alone)	Car ◇— Engine
Composition	filled diamond at the whole	is entirely made of (parts die with whole)	Book ◆— Page

2. Intent and problem (sl. 4)

> **MEMORIZE** **Factory Method** — Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.

Problem: a framework standardises the architecture for a range of applications, but each application defines its own domain objects and their creation. The creator defers product creation to a subclass.

3. Structure and participants (sl. 5)



Participant	Role
Product	Interface of the objects the factory method creates
ConcreteProduct	Implements Product
Creator	Declares factoryMethod() returning Product; its other code works only with Product
ConcreteCreator	Overrides factoryMethod() to return one ConcreteProduct

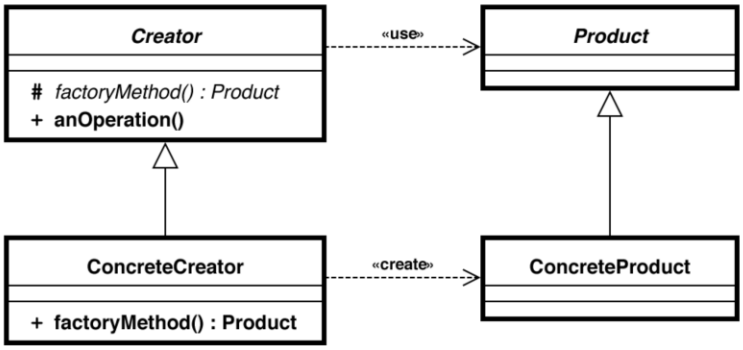


Figure 5.1 — Factory Method structure (sl. 5)

4. Slide examples (sl. 6–14)

- **Drawing (sl. 6–7):** Shape interface (draw), Circle/Square/Rectangle; ShapeFactory.getShape(type) returns the right shape; the client asks the factory instead of calling new. (This parameterised version is often called a 'simple factory'.)
- **Kiosk (sl. 8):** PaymentProcessor (processPayment) with Cash, CreditCard, Coupon; GetPayment.getProcessor(type).
- **Electricity bill (sl. 9–11):** abstract Plan(rate; getRate(); calculateBill(units) = units × rate); DomesticPlan 3.50, CommercialPlan 7.50, InstitutionalPlan 5.50; GetPlanFactory.getPlan(planType); GenerateBill reads plan + units and prints the bill. E.g. 100 commercial units → 100 × 7.50 = 750.
- **Logistics (sl. 12–13):** code coupled to Truck; adding Ship would touch everything. Logistics.createTransport() is overridden by RoadLogistics (Truck) and SeaLogistics (Ship); planDelivery() works with Transport. Dialog.createButton(): WindowsDialog → WindowsButton, WebDialog → HTMLButton.
- **UI framework (sl. 14):** to make the framework use RoundButton, subclass UIFramework as UIWithRoundButtons and override createButton(). Use Factory Method when users of a library must be able to extend its internal components.

5. Consequences (sl. 15–16)

- Code for an interface, not an implementation; instantiation is removed from client code → less coupling, easy to extend.
- Single Responsibility (creation in one place) and Open/Closed (new products without breaking clients).
- Cost: many new subclasses (one creator per product).

Exam Approach

1. Cue words: 'client must not construct the concrete class', 'centralised creation', 'add new types without modifying'.
2. Draw Product interface + concrete products, Creator with abstract factory method + concrete creators.
3. Write the interface, two products, the creator hierarchy and a main() that uses only the abstract types.
4. End with one line on extension: 'new type = new product + new creator; no existing class changes' (OCP).

Practice — Lesson 5

(a) Actual — 2024-25 (22 batch) Q1(a). (b) Actual — 2016-17 Q5(c). (c) Actual — 2010-11 Q3(d).

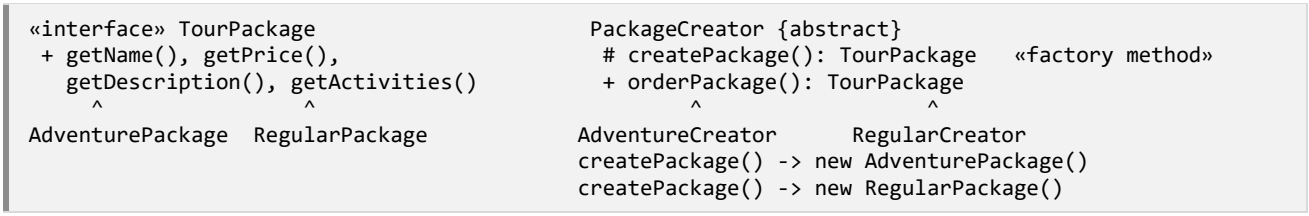
(a) A travel company offers Adventure and Regular packages, each with its own activities, pricing and description. Implement it with a pattern so that client code never constructs a concrete class, packages are created through a centralised creation component, and new package types need no change to existing code. (15)

(b) Syntax highlighting: choose a parser (.c, .cpp, .java, .html …), all implementing Parser, from the file extension. Which pattern? Give a Java implementation. (3+7)

(c) XMLReaderFactory has a static createXMLReader() that returns an XMLReader implementation chosen from a system property. Does it implement the Factory pattern? (5)

✓ Solution

(a) Factory Method. The creator PackageCreator owns the creation step (createPackage()); each concrete creator decides the class. Client code holds only PackageCreator and TourPackage references.



```
// 2024-25 Q1(a) — Factory Method: client never names a concrete package class
import java.util.List;

interface TourPackage {
    String getName();
    double getPrice();
    String getDescription();
    List<String> getActivities();
}
class AdventurePackage implements TourPackage {
    public String getName() { return "Adventure"; }
    public double getPrice() { return 25000; }
    public String getDescription() { return "For thrill-seekers"; }
    public List<String> getActivities() { return List.of("Mountain hiking", "River rafting"); }
}
class RegularPackage implements TourPackage {
    public String getName() { return "Regular"; }
    public double getPrice() { return 12000; }
    public String getDescription() { return "For leisure travellers"; }
    public List<String> getActivities() { return List.of("City sightseeing", "Museum visits"); }
}

// Creator = the centralised creation component
abstract class PackageCreator {
    protected abstract TourPackage createPackage();          // the factory method
    public TourPackage orderPackage() {                       // common logic works on the interface
        TourPackage p = createPackage();
        System.out.println("Booked " + p.getName() + " (" + p.getPrice() + "): " +
p.getActivities());
        return p;
    }
}
class AdventureCreator extends PackageCreator {
    protected TourPackage createPackage() { return new AdventurePackage(); }
}
class RegularCreator extends PackageCreator {
    protected TourPackage createPackage() { return new RegularPackage(); }
}
// New type later: HoneymoonPackage + HoneymoonCreator. No existing class is edited.

public class TravelDemo {
    public static void main(String[] args) {
        PackageCreator creator = new AdventureCreator();   // chosen by configuration / UI
        creator.orderPackage();
        new RegularCreator().orderPackage();
    }
}
```

Output: Booked Adventure (25000.0): [Mountain hiking, River rafting] / Booked Regular (12000.0): [City sightseeing, Museum visits].

(b) Factory (Factory Method family): the selection logic is one creation component that maps an extension to a Parser; the editor uses only the Parser interface. Assumptions: extension = text after the last dot; unknown extension → exception. A registry map keeps the factory closed for modification (new language = register()).

```
// 2016-17 Q5(c) — choose a parser from the file extension (factory with a registry)
import java.util.HashMap;
import java.util.Map;
import java.util.function.Supplier;

interface Parser { void parse(String fileName); }
class CParser    implements Parser { public void parse(String f) { System.out.println("C parser: " +
f); } }
class CppParser  implements Parser { public void parse(String f) { System.out.println("C++ parser: "
+ f); } }
class JavaParser implements Parser { public void parse(String f) { System.out.println("Java parser: "
+ f); } }
class HtmlParser implements Parser { public void parse(String f) { System.out.println("HTML parser: "
+ f); } }

class ParserFactory {
    private static final Map<String, Supplier<Parser>> REG = new HashMap<>();
    static {
        REG.put("c", CParser::new);
        REG.put("cpp", CppParser::new);
        REG.put("java", JavaParser::new);
        REG.put("html", HtmlParser::new);
    }
    static void register(String ext, Supplier<Parser> s) { REG.put(ext, s); }  // new language
    static Parser getParser(String fileName) {
        String ext = fileName.substring(fileName.lastIndexOf('.') + 1).toLowerCase();
        Supplier<Parser> s = REG.get(ext);
        if (s == null) throw new IllegalArgumentException("No parser for ." + ext);
        return s.get();
    }
}
public class ParserDemo {
    public static void main(String[] args) {
        for (String f : new String[]{"main.c", "a.cpp", "App.java", "index.html"})
            ParserFactory.getParser(f).parse(f);
    }
}
```

(c) Yes. createXMLReader() is a static factory method: it returns an object of some class implementing XMLReader, the concrete class is chosen inside the method (from a system property), and the client depends only on the XMLReader interface. Strictly it is the 'simple/static factory' variant — no subclass overrides a creator method — so it is the Factory idea, not the full GoF Factory Method structure.

Lesson 6 — Abstract Factory

Slides 17–35 (Lec 3 - Creational Design Patterns)

> **MEMORIZE** **Abstract Factory** — Provide an interface for creating **families** of related or dependent objects without specifying their concrete classes.

Problem (sl. 17): a portable application must encapsulate platform dependencies — several look-and-feels, several DBMSs.

1. Structure (sl. 18)

```
«interface» AbstractFactory              «interface» ProductA      «interface» ProductB
 + createProductA(): ProductA                  ^          ^              ^          ^
 + createProductB(): ProductB               ProductA1  ProductA2      ProductB1  ProductB2
        ^                ^
 ConcreteFactory1    ConcreteFactory2      Client --> AbstractFactory, ProductA, ProductB only
 createA()->A1        createA()->A2
 createB()->B1        createB()->B2        (factory 1 always makes family 1: A1 + B1)
```

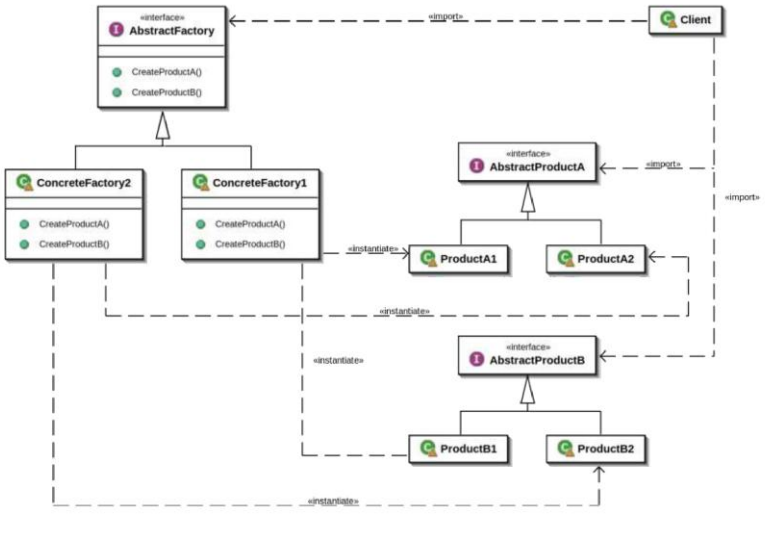


Figure 6.1 — Abstract Factory structure (sl. 18)

2. Slide examples (sl. 19–33)

- **Furniture shop (sl. 19–22):** products Chair, Sofa, CoffeeTable × variants ArtDeco, Victorian, Modern. Customers get angry with non-matching furniture. Declare an interface per product, then FurnitureFactory (createChair, createSofa, createCoffeeTable); ModernFurnitureFactory makes only Modern pieces.
- **Cross-platform GUI (sl. 23–29):** GUIFactory (createButton, createCheckbox) with WinFactory and MacFactory. ApplicationConfigurator reads the OS at start-up and passes the right factory to Application; the rest of the code works with Button and Checkbox interfaces only.
- **Car factory (sl. 30):** HondaFactory (createEngine, createTrans) with CivicFactory and AccordFactory → CivicEngine/CivicTrans or AccordEngine/AccordTrans.
- **Factory of factories (sl. 31–33):** AbstractFactory (getShape, getColor), ShapeFactory, ColorFactory, FactoryProducer.getFactory("SHAPE" | "COLOR").

3. Consequences (sl. 34–35)

Benefit	Liability

Benefit	Liability
Isolates concrete classes; client sees only interfaces	Supporting a **new kind of product** is hard: every factory interface and class must get a new create method
Exchanging a whole product family is easy (swap one factory object)	More interfaces and classes
Promotes consistency — products of one family are used together	
No conditional logic in the client (unlike a parameterised factory)	

TRAP Factory Method creates **one** product through inheritance (subclass overrides one method). Abstract Factory creates a **family** through object composition (the client is given a factory object). Each create method inside an Abstract Factory is often a factory method.

Exam Approach

1. Cue words: 'families', 'region/platform/brand-specific', 'must match', 'consistent', 'sensitive to their orders', 'easily extendable to new countries/platforms'.
2. Make a table: rows = families (countries, menus, companies), columns = product kinds (crust/sauce, rice/main/fry, car/moto).
3. One interface per column; one concrete factory per row with one create method per column.
4. Client receives the factory once and never names a concrete product.

Practice — Lesson 6

(a) Actual — 2023-24 Q5(a). (b) Actual — 2021-22 Q3(a). (c) Actual — 2017-18 Q7(b). (d) Actual — 2014-15 Q6(b).

(a) Pizza delivery in Italy and America with distinct crust and toppings; each pizza must use region-specific ingredients and the system must extend easily to more countries. Identify the pattern, justify, and implement. (15)

(b) Four set menus (Veggie Harvest, Chicken Delight, Beef Fiesta, Sea Shanty), each a rice item + a main dish + a fried item made in kitchens Rice Station, Main Mania and Fry More; users are very sensitive to their orders. Identify the pattern, give the UML and an implementation. (20)

(c) Uber and Pathao compute car fares n·d·18·f (f = 1.8/1.2/0.9) and n·d·20·f (f = 1.5/1.15/0.9), and moto fares 25 + 12d and 30 + 11d. Input distance, passengers and t (1, 2, 3); output the cost for each app. Class diagram + code. (6+15)

(d) Give one design pattern that reduces dependence on hardware and/or software platforms and explain how. (12)

✓ Solution

(a) Abstract Factory. Each country is a family of matching ingredients (crust, sauce, topping). The pizza code receives a PizzaIngredientFactory and cannot accidentally mix an Italian crust with an American topping; a new country is one new factory class plus its ingredient classes, with no change to RegionalPizza (OCP).

```
PizzaIngredientFactory «interface»: createCrust(), createSauce(), createTopping()
   ^ ItalyIngredientFactory   -> ThinCrust,     MarinaraSauce,   Mozzarella
   ^ AmericaIngredientFactory -> DeepDishCrust, PlumTomatoSauce, Pepperoni
Crust, Sauce, Topping «interface»     RegionalPizza(f: PizzaIngredientFactory) -- uses --> the three
interfaces
```

```
// 2023-24 Q5(a) — Abstract Factory: each region creates a consistent family of ingredients
interface Crust   { String name(); }
interface Sauce   { String name(); }
interface Topping { String name(); }

class ThinCrust       implements Crust   { public String name() { return "thin crust"; } }
class MarinaraSauce   implements Sauce   { public String name() { return "marinara"; } }
class Mozzarella      implements Topping { public String name() { return "fresh mozzarella, basil"; }
}
class DeepDishCrust   implements Crust   { public String name() { return "deep-dish crust"; } }
class PlumTomatoSauce implements Sauce   { public String name() { return "plum-tomato sauce"; } }
class Pepperoni       implements Topping { public String name() { return "cheddar, pepperoni"; } }

interface PizzaIngredientFactory {                // abstract factory
    Crust createCrust();
    Sauce createSauce();
    Topping createTopping();
}
class ItalyIngredientFactory implements PizzaIngredientFactory {
    public Crust createCrust()     { return new ThinCrust(); }
    public Sauce createSauce()     { return new MarinaraSauce(); }
    public Topping createTopping() { return new Mozzarella(); }
}
class AmericaIngredientFactory implements PizzaIngredientFactory {
    public Crust createCrust()     { return new DeepDishCrust(); }
    public Sauce createSauce()     { return new PlumTomatoSauce(); }
    public Topping createTopping() { return new Pepperoni(); }
}
class RegionalPizza {                             // client: never names a concrete ingredient
    private final Crust crust; private final Sauce sauce; private final Topping topping;
    RegionalPizza(PizzaIngredientFactory f) {
        crust = f.createCrust(); sauce = f.createSauce(); topping = f.createTopping();
    }
    public String toString() { return crust.name() + " + " + sauce.name() + " + " + topping.name(); }
}
public class PizzaDemo {
    static PizzaIngredientFactory factoryFor(String country) {   // chosen once
        switch (country) {
            case "Italy":   return new ItalyIngredientFactory();
            case "America": return new AmericaIngredientFactory();
            default: throw new IllegalArgumentException(country);
        }
    }
    public static void main(String[] args) {
        System.out.println("Italy:   " + new RegionalPizza(factoryFor("Italy")));
        System.out.println("America: " + new RegionalPizza(factoryFor("America")));
    }
}
```

(b) Abstract Factory. Rows = set menus (families), columns = kitchens (product kinds). 'Users are very sensitive to their orders' means a menu's three items must come from the same family — exactly the consistency Abstract Factory guarantees. (A Builder would assemble one order, but would not stop a builder step from mixing items of two menus.)

```
SetMenuFactory «interface»: prepareRice(): RiceItem, prepareMain(): MainDish, prepareFried():
FriedItem
  ^ VeggieHarvestFactory   Mushroom Rice      | Paneer Masala    | Fried Vegetable Pakora
  ^ ChickenDelightFactory  Chicken Fried Rice | Butter Chicken   | Fried Chicken Wings
  ^ BeefFiestaFactory      Beef Fried Rice    | Beef Chili Onion | Fried Beef Bacon
  ^ SeaShantyFactory       Rice with Shrimp   | Crab Masala      | Fried Calamari
Restaurant.deliver(f: SetMenuFactory) — the client
```

```
// 2021-22 Q3(a) — Abstract Factory: one factory per set menu; one product type per kitchen
interface RiceItem  { String name(); }   // made in Rice Station
interface MainDish  { String name(); }   // made in Main Mania
interface FriedItem { String name(); }   // made in Fry More

class Dish implements RiceItem, MainDish, FriedItem {
    private final String n;
    Dish(String n) { this.n = n; }
    public String name() { return n; }
}
interface SetMenuFactory {
    RiceItem prepareRice();
    MainDish prepareMain();
    FriedItem prepareFried();
}
class VeggieHarvestFactory implements SetMenuFactory {
    public RiceItem prepareRice()   { return new Dish("Mushroom Rice"); }
    public MainDish prepareMain()   { return new Dish("Paneer Masala"); }
    public FriedItem prepareFried() { return new Dish("Fried Vegetable Pakora"); }
}
class ChickenDelightFactory implements SetMenuFactory {
    public RiceItem prepareRice()   { return new Dish("Chicken Fried Rice"); }
    public MainDish prepareMain()   { return new Dish("Butter Chicken"); }
    public FriedItem prepareFried() { return new Dish("Fried Chicken Wings"); }
}
class BeefFiestaFactory implements SetMenuFactory {
    public RiceItem prepareRice()   { return new Dish("Beef Fried Rice"); }
    public MainDish prepareMain()   { return new Dish("Beef Chili Onion"); }
    public FriedItem prepareFried() { return new Dish("Fried Beef Bacon"); }
}
class SeaShantyFactory implements SetMenuFactory {
    public RiceItem prepareRice()   { return new Dish("Rice with Shrimp"); }
    public MainDish prepareMain()   { return new Dish("Crab Masala"); }
    public FriedItem prepareFried() { return new Dish("Fried Calamari"); }
}
class Restaurant {                 // client: assembles one family; cannot mix menus by mistake
    String deliver(SetMenuFactory f) {
        return f.prepareRice().name() + ", " + f.prepareMain().name() + ", " +
f.prepareFried().name();
    }
}
```

```
public class SetMenuDemo {
    public static void main(String[] args) {
        Restaurant r = new Restaurant();
        SetMenuFactory[] menus = { new VeggieHarvestFactory(), new ChickenDelightFactory(),
                                   new BeefFiestaFactory(), new SeaShantyFactory() };
        for (SetMenuFactory m : menus)
            System.out.println(m.getClass().getSimpleName().replace("Factory", "") + ": " +
r.deliver(m));
    }
}
```

(c) Abstract Factory: families = companies (Uber, Pathao); products = CarFare and MotoFare. Each fare object holds its company's formula. (Strategy for the fare formula is also acceptable; Abstract Factory is the better answer because each company supplies a matching pair.)

```
// 2017-18 Q7(b) — Abstract Factory: each company supplies a matching (car, moto) fare family
interface CarFare  { double cost(int passengers, double d, int t); }
interface MotoFare { double cost(double d); }

class UberCar implements CarFare {
    public double cost(int n, double d, int t) {
        double f = (t == 1) ? 1.8 : (t == 2) ? 1.2 : 0.9;
        return n * d * 18 * f;
    }
}
class PathaoCar implements CarFare {
    public double cost(int n, double d, int t) {
        double f = (t == 1) ? 1.5 : (t == 2) ? 1.15 : 0.9;
        return n * d * 20 * f;
    }
}
class UberMoto   implements MotoFare { public double cost(double d) { return 25 + d * 12; } }
class PathaoMoto implements MotoFare { public double cost(double d) { return 30 + d * 11; } }

interface RideFactory { CarFare carFare(); MotoFare motoFare(); String name(); }
class UberFactory implements RideFactory {
    public CarFare carFare()   { return new UberCar(); }
    public MotoFare motoFare() { return new UberMoto(); }
    public String name()       { return "Uber"; }
}
class PathaoFactory implements RideFactory {
    public CarFare carFare()   { return new PathaoCar(); }
    public MotoFare motoFare() { return new PathaoMoto(); }
    public String name()       { return "Pathao"; }
}
public class RideDemo {
    public static void main(String[] args) {
        double d = 10; int n = 2; int t = 1;   // in the exam read these with a Scanner
        for (RideFactory f : new RideFactory[]{ new UberFactory(), new PathaoFactory() })
            System.out.printf("%s car: %.2f   %s moto: %.2f%n",
                    f.name(), f.carFare().cost(n, d, t), f.name(), f.motoFare().cost(d));
    }
}
```

Check, d = 10 km, n = 2, t = 1: Uber car 2 × 10 × 18 × 1.8 = 648; Pathao car 2 × 10 × 20 × 1.5 = 600; Uber moto 25 + 120 = 145; Pathao moto 30 + 110 = 140 — matches the program output.

(d) Abstract Factory (sl. 17, 23–29). All platform-specific classes (WinButton, MacButton …) are created only inside one concrete factory. At start-up the configurator inspects the OS and creates WinFactory or MacFactory; the rest of the program talks to GUIFactory, Button and Checkbox interfaces. Porting to a new platform = writing one new factory and its products; no application code changes. (Bridge is a valid second answer: it separates the abstraction from a platform implementation hierarchy — IJ Mam's notes.)

Lesson 7 — Singleton

Slides 36–42 (Lec 3 - Creational Design Patterns)

MEMORIZE **Singleton** — Ensure a class has only one instance, and provide a global point of access to it. Encapsulated 'just-in-time initialisation' (initialisation on first use).

1. Structure (sl. 37) and implementation steps (sl. 39)

```
         Singleton
 --------------------------------
 - instance: Singleton            (static)
 --------------------------------
 - Singleton()                    (private constructor)
 + getInstance(): Singleton       (static)  { if (instance == null) instance = new Singleton();
return instance; }
```

1. Add a private static field to hold the instance.
2. Declare a public static creation method getInstance().
3. Implement lazy initialisation inside it: create on the first call, return the same object afterwards.
4. Make the constructor private so other code cannot use new.
5. Replace direct constructor calls in client code with getInstance().

Slide example (sl. 38): DbConnection with private static instance and a private constructor that opens the SQL connection; DbConnection.getDbConnection() returns the shared connection.

2. Applicability and consequences (sl. 40–42)

- Use it when a class must have exactly one instance available to all clients (database connection, configuration, logger).
- Unlike a global variable, nothing except the class itself can replace the cached instance.
- Consequences: controlled access to the sole instance; reduced name space (no globals); permits a variable number of instances — the same code can be changed to allow n instances (used in 2011-12 Q1(b) below).
- Criticism: it violates SRP (it solves two problems at once) and makes unit testing harder.

3. Eager vs lazy initialisation (textbook — asked 2016-17, 2012-13)

	Eager	Lazy
When created	When the class is loaded (static final field)	On the first getInstance() call
Thread safety	Safe automatically (class loading is thread-safe)	Needs synchronized / double-checked locking / holder class
Cost	Wastes memory/time if never used	Pays only when used; small check on every call
Prefer when	Object is cheap and always needed	Object is expensive (DB connection, large cache) or may never be used

```
// 2016-17 Q5(a), 2012-13 Q2(a), 2010-11 Q4(a) – eager, lazy, double-checked, holder
class EagerSingleton {
    private static final EagerSingleton INSTANCE = new EagerSingleton();  // made at class load
    private EagerSingleton() { }
    public static EagerSingleton getInstance() { return INSTANCE; }
}
class LazySingleton {
    private static LazySingleton instance;                                // made on first use
    private LazySingleton() { }
    public static synchronized LazySingleton getInstance() {
        if (instance == null) instance = new LazySingleton();
        return instance;
    }
}
class DclSingleton {
    private static volatile DclSingleton instance;
    private DclSingleton() { }
    public static DclSingleton getInstance() {
        if (instance == null) {
            synchronized (DclSingleton.class) {
                if (instance == null) instance = new DclSingleton();
            }
        }
        return instance;
    }
}
class HolderSingleton {                                  // 2010-11 Q4(a)(ii): valid, lazy, thread-safe
    private HolderSingleton() { }
    private static class SingletonHolder {
        private static final HolderSingleton instance = new HolderSingleton();
    }
    public static HolderSingleton getInstance() { return SingletonHolder.instance; }
}
public class SingletonKinds {
    public static void main(String[] args) {
        System.out.println(EagerSingleton.getInstance() == EagerSingleton.getInstance());
        System.out.println(LazySingleton.getInstance() == LazySingleton.getInstance());
        System.out.println(DclSingleton.getInstance() == DclSingleton.getInstance());
        System.out.println(HolderSingleton.getInstance() == HolderSingleton.getInstance());
    }
}
```

Practice — Lesson 7

(a) Actual — 2019-20 Q1(a). (b) Actual — 2016-17 Q5(a) / 2012-13 Q2(a). (c) Actual — 2010-11 Q4(a). (d) Actual — 2011-12 Q1(b).

(a) A web app creates userID, home-district, qualification and transaction (TxID) identifiers in the back end, each never duplicated and created serially from 1, used as primary keys. Demonstrate their creation with a suitable pattern. (15)

(b) Describe eager and lazy initialisation of a singleton; give an example where lazy should be preferred; draw the structure. (15)

(c) Which of the two implementations is a valid Singleton? (i) eager field but getInstance() is private; (ii) SingletonHolder nested class with public getInstance(). (5)

(d) Class Base is inherited by Derived1 and Derived2; at most 5 Derived1 and 10 Derived2 objects can exist. Write the Java code and a main that creates arrays of both. (18⅔)

✓ Solution

(a) Singleton (one generator object per identifier kind). If two generator objects existed, both would start at 1 and hand out duplicate primary keys; a single shared instance with a synchronized counter guarantees serial, unique values for the whole back end. In a real system the counter would be initialised from MAX(id) in the table at start-up.

```
// 2019-20 Q1(a) — one Singleton generator per identifier kind; serial ids from 1, never duplicated
class IdGenerator {
    private static IdGenerator userIds, districtIds, qualificationIds, txIds;
    private long next = 1;
    private IdGenerator() { }                                  // no outside construction

    public static synchronized IdGenerator forUser() {
        if (userIds == null) userIds = new IdGenerator();
        return userIds;
    }
    public static synchronized IdGenerator forDistrict() {
        if (districtIds == null) districtIds = new IdGenerator();
        return districtIds;
    }
    public static synchronized IdGenerator forQualification() {
        if (qualificationIds == null) qualificationIds = new IdGenerator();
        return qualificationIds;
    }
    public static synchronized IdGenerator forTransaction() {
        if (txIds == null) txIds = new IdGenerator();
        return txIds;
    }
    public synchronized long nextId() { return next++; }      // strictly serial
}
class RegistrationService {                // back end called by the UI
    long registerUser(String name)      { return IdGenerator.forUser().nextId(); }
    long createDistrict(String d)       { return IdGenerator.forDistrict().nextId(); }
    long createQualification(String q)  { return IdGenerator.forQualification().nextId(); }
    long recordTransaction()            { return IdGenerator.forTransaction().nextId(); }
}
public class IdDemo {
    public static void main(String[] args) {
        RegistrationService s = new RegistrationService();
        System.out.println("user " + s.registerUser("A") + ", user " + s.registerUser("B"));
        System.out.println("district " + s.createDistrict("Dhaka") + ", tx " + s.recordTransaction()
                + ", tx " + s.recordTransaction() + ", qualification " +
s.createQualification("BSc"));
        System.out.println("same object? " + (IdGenerator.forUser() == IdGenerator.forUser()));
    }
}
```

Output: user 1, user 2 / district 1, tx 1, tx 2, qualification 1 / same object? true.

(b) Use the table in §3 and the code above. Lazy preferred example: a DatabaseConnectionPool or report generator that opens many connections — creating it at start-up slows launch and wastes resources if the user never opens a report. Structure: the box in §1.

(c) (i) is **not usable**: the eager field is fine, but getInstance() is private, so no other class can obtain the instance — there is no global access point. (ii) is **valid** (initialisation-on-demand holder): the private constructor stops new, the nested class is loaded only when getInstance() is first called (lazy), and the JVM's class loading makes it thread-safe without synchronized.

(d) Generalise the private constructor + static access method so the class counts its instances and refuses the (limit+1)-th one.

```
// 2011-12 Q1(b) — Base inherited by Derived1 (max 5) and Derived2 (max 10): Singleton generalised
import java.util.HashMap;
import java.util.Map;

abstract class Base {
    private static final Map<Class<?>, Integer> COUNT = new HashMap<>();
    protected Base(int limit) {                       // each subclass passes its own limit
        int c = COUNT.getOrDefault(getClass(), 0);
        if (c >= limit)
            throw new IllegalStateException(getClass().getSimpleName() + ": limit " + limit + "
reached");
        COUNT.put(getClass(), c + 1);
    }
}
class Derived1 extends Base {
    private static final int MAX = 5;
    private Derived1() { super(MAX); }                // private: only getInstance() creates
    public static synchronized Derived1 getInstance() {
        try { return new Derived1(); } catch (IllegalStateException e) { return null; }
    }
}
class Derived2 extends Base {
    private static final int MAX = 10;
    private Derived2() { super(MAX); }
    public static synchronized Derived2 getInstance() {
        try { return new Derived2(); } catch (IllegalStateException e) { return null; }
    }
}
public class LimitedDemo {
    public static void main(String[] args) {
        Derived1[] a = new Derived1[7];
        Derived2[] b = new Derived2[12];
        int ok1 = 0, ok2 = 0;
        for (int i = 0; i < a.length; i++) if ((a[i] = Derived1.getInstance()) != null) ok1++;
        for (int i = 0; i < b.length; i++) if ((b[i] = Derived2.getInstance()) != null) ok2++;
        System.out.println("Derived1 created: " + ok1 + " of " + a.length);
        System.out.println("Derived2 created: " + ok2 + " of " + b.length);
    }
}
```

Output: Derived1 created: 5 of 7 / Derived2 created: 10 of 12.

Lesson 8 — Builder and Choosing a Creational Pattern

Slides 43–53 (Lec 3 - Creational Design Patterns)

> **MEMORIZE** **Builder** — Separate the construction of a complex object from its representation so that the same construction process can create different representations.

Problem (sl. 43): an application builds the parts of a complex aggregate; one construction specification must be able to produce several different representations.

1. Structure (sl. 44)

```
  Director                      «interface» Builder                Product
- builder: Builder  <>---------> + buildPartA()                 (complex object,
+ construct() {                    + buildPartB()                 assembled in parts)
    builder.buildPartA();           + getResult(): Product              ^
    builder.buildPartB(); }               ^          ^                  | creates
                                  ConcreteBuilder1  ConcreteBuilder2 ----+
```

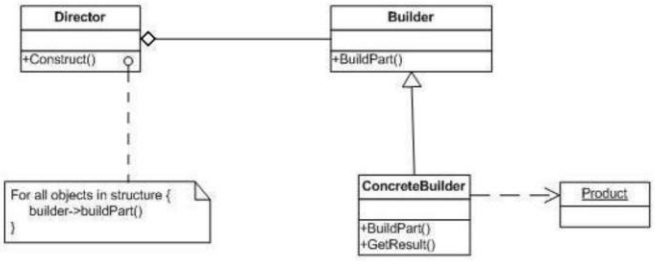


Figure 8.1 — Builder structure (sl. 44)

Participant	Role (kids-meal analogy, sl. 45)
Director	Fixes the order of the steps (the cashier tells the crew: burger, drink, toy)
Builder	Interface with one method per part
ConcreteBuilder	Builds and remembers one representation (the restaurant crew for one meal type)
Product	The finished complex object (the bagged meal)
Client	Picks the concrete builder, gives it to the director, collects the product (the customer)

2. Slide examples (sl. 45–53)

- Computer assembled from parts per order (500 GB disk + Intel vs 250 GB disk + AMD).
- Vehicle (sl. 47–49): IBuilder (BuildBody, InsertWheels, AddHeadlights, GetVehicle); Car and MotorCycle builders; Product keeps a list of parts and Show(); Director.Construct(builder) calls the three steps in order.
- CD packing (sl. 50–53): Packing (pack, price); CD → Company → Sony (20), Samsung (15); CDType holds items (addItem, getCost, showItems); CDBuilder.buildSonyCD() / buildSamsungCD(); BuilderDemo prints the costs.

3. Consequences (sl. 46)

- Lets you vary a product's internal representation.
- Isolates construction code from representation code.
- Gives finer control over the construction process (step by step, under the director).

4. Choosing a creational pattern (2021-22 Q1(a))

Pattern	Use when	Example
Factory Method	One product; the subclass (or a parameter) decides which concrete class; client must not call new	Logistics.createTransport() → Truck / Ship; travel packages
Abstract Factory	A family of related products must be created together and stay consistent; swap the family as a whole	GUIFactory → WinButton + WinCheckbox; pizza ingredients per country
Builder	One complex object built step by step; same steps give different representations; many optional parts	HTML page; meals; forms per user type
Singleton	Exactly one shared instance with global access	DB connection, id generator
Bridge (structural)	Two independent dimensions (abstraction × implementation) would explode into subclasses; connect two hierarchies by composition	Shape × Color; Notification × Email/SMS/Push

Practice — Lesson 8

(a) Actual — 2022-23 Q3(a). (b) Actual — 2018-19 Q3(b). (c) Actual — 2017-18 Q6(a). (d) Actual — 2020-21 Q1(a)–(b). (e) Actual — 2021-22 Q1(a).

(a) HTMLPageDirector tells the builder HTMLPageBuilder the pageTitle, pageHeading and several lines of text, then has it assemble the parts and return the page. Class diagram + code. (15)

(b) A grades application shows different forms (enabled/disabled buttons) to admin and student. Builder gives the form interface, concrete builders are the per-user forms, the product is the final form and the director is the application. Class diagram + code. (20)

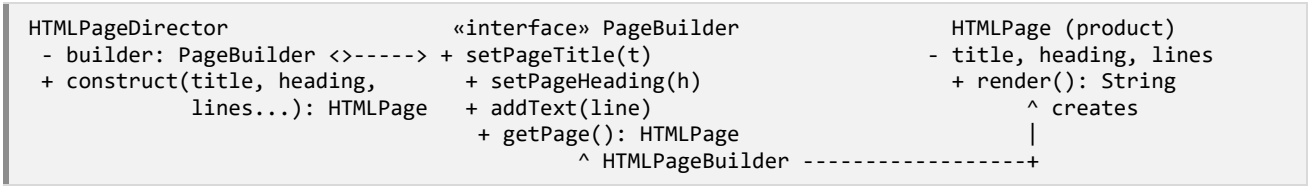
(c) Lalbagh food shop: set meals with a main item (rice or bread); rice comes with gravy curries, bread with chicken or beef kebabs; drinks are ordered separately; the bill must be paid before eating. Class diagram + code. (7+15)

(d) Burger shop: Veggi/Beef/Chicken pizza, optional extra cheese, drinks (Coffee, Water, Coke — one or more), appetizers (Onion Rings, French Fries), and meals (appetizer + pizza with cheese + drinks). Construct the four orders (1 Beef pizza + French fry + cheese, 2 Chicken pizza + onion rings + water, 3 combo Veggi + French fry + two Cokes, 4 combo Veggi + Onion Rings + Coffee + Water) using three patterns, with price methods; then compare the patterns. (28+7)

(e) Differentiate the use cases of Factory Method, Abstract Factory, Builder and Bridge. (16)

✓ Solution

(a) Builder with the names given in the question.



```
// 2022-23 Q3(a) — Builder: HTMLPageDirector drives HTMLPageBuilder
import java.util.ArrayList;
import java.util.List;

class HTMLPage {                                        // product
    private String title = "", heading = "";
    private final List<String> lines = new ArrayList<>();
    void setTitle(String t)   { title = t; }
    void setHeading(String h) { heading = h; }
    void addLine(String l)    { lines.add(l); }
    String render() {
        StringBuilder sb = new StringBuilder("<html><head><title>" + title +
"</title></head><body>\n");
        sb.append("<h1>").append(heading).append("</h1>\n");
        for (String l : lines) sb.append("<p>").append(l).append("</p>\n");
        return sb.append("</body></html>").toString();
    }
}
interface PageBuilder {                                 // builder
    void setPageTitle(String t);
    void setPageHeading(String h);
    void addText(String line);
    HTMLPage getPage();                                 // final assembly
}
class HTMLPageBuilder implements PageBuilder {          // concrete builder
    private final HTMLPage page = new HTMLPage();
    public void setPageTitle(String t)   { page.setTitle(t); }
    public void setPageHeading(String h) { page.setHeading(h); }
    public void addText(String line)     { page.addLine(line); }
    public HTMLPage getPage()            { return page; }
}
class HTMLPageDirector {                                // director
    private final PageBuilder builder;
    HTMLPageDirector(PageBuilder b) { builder = b; }
    HTMLPage construct(String title, String heading, String... lines) {
        builder.setPageTitle(title);
        builder.setPageHeading(heading);
        for (String l : lines) builder.addText(l);
        return builder.getPage();
    }
}
public class HtmlDemo {
    public static void main(String[] args) {
        HTMLPageDirector d = new HTMLPageDirector(new HTMLPageBuilder());
        System.out.println(d.construct("CSE 213", "Builder Pattern", "Line one.", "Line
two.").render());
    }
}
```

(b) Builder exactly as the question maps it: FormBuilder (interface/abstract), AdminFormBuilder and StudentFormBuilder (concrete builders), Form (product), GradeApplication (director, chooses the builder from the login role and calls the steps in a fixed order).

```
// 2018-19 Q3(b) — Builder: a form per user type; Director = the application
import java.util.ArrayList;
import java.util.List;

class Form {                                                    // product
    private final List<String> parts = new ArrayList<>();
    void add(String p) { parts.add(p); }
    public String toString() { return String.join(" | ", parts); }
}
abstract class FormBuilder {                                    // builder
    protected Form form;
    void createNewForm() { form = new Form(); }
    abstract void buildHeader();
    abstract void buildGradeTable();
    abstract void buildButtons();
    Form getForm() { return form; }
}
class AdminFormBuilder extends FormBuilder {                    // concrete builders
    void buildHeader()     { form.add("Header: Admin panel"); }
    void buildGradeTable() { form.add("Grades: all students (editable)"); }
    void buildButtons()    { form.add("Edit[on] Publish[on] Delete[on]"); }
}
class StudentFormBuilder extends FormBuilder {
    void buildHeader()     { form.add("Header: My grades"); }
    void buildGradeTable() { form.add("Grades: own (read-only)"); }
    void buildButtons()    { form.add("Edit[off] Publish[off] Print[on]"); }
}
class GradeApplication {                                        // director
    Form formFor(String role) {
        FormBuilder b = role.equals("admin") ? new AdminFormBuilder() : new StudentFormBuilder();
        b.createNewForm();
        b.buildHeader();
        b.buildGradeTable();
        b.buildButtons();
        return b.getForm();
    }
}
public class FormDemo {
    public static void main(String[] args) {
        GradeApplication app = new GradeApplication();
        System.out.println("admin:   " + app.formFor("admin"));
        System.out.println("student: " + app.formFor("student"));
    }
}
```

(c) Builder: every set meal is built by the same steps (main item, sides, optional drink, bill) but each concrete builder gives a different representation. Waiter = director; the bill is computed from the product before it is served. Prices are assumed.

```
Waiter (director) <>--> SetMealBuilder {abstract}: addMainItem(), addSides(), addDrink(n,p),
getMeal()
                              ^ RiceMealBuilder     (rice + gravy curry + dal)
                              ^ ChickenKebabMealBuilder (naan + chicken kebab)
                              ^ BeefKebabMealBuilder    (naan + beef kebab)
SetMeal (product) — list of Food; bill()
```

```
// 2017-18 Q6(a) — Builder: set meals (rice + curries, or bread + kebabs), optional drinks, bill
first
import java.util.ArrayList;
import java.util.List;

class Food {
    final String name; final double price;
    Food(String name, double price) { this.name = name; this.price = price; }
}
class SetMeal {                                                  // product
    private final List<Food> items = new ArrayList<>();
    void add(Food f) { items.add(f); }
    double bill() { double s = 0; for (Food f : items) s += f.price; return s; }
    void show() {
        for (Food f : items) System.out.printf("  %-20s %6.1f%n", f.name, f.price);
        System.out.printf("  %-20s %6.1f%n", "BILL (pay first)", bill());
    }
}
abstract class SetMealBuilder {                                  // builder
    protected final SetMeal meal = new SetMeal();
    abstract void addMainItem();
    abstract void addSides();
    void addDrink(String name, double price) { meal.add(new Food(name, price)); }
    SetMeal getMeal() { return meal; }
}
class RiceMealBuilder extends SetMealBuilder {
    void addMainItem() { meal.add(new Food("Plain rice", 40)); }
    void addSides()    { meal.add(new Food("Chicken gravy curry", 90)); meal.add(new Food("Dal",
20)); }
}
class ChickenKebabMealBuilder extends SetMealBuilder {
    void addMainItem() { meal.add(new Food("Naan x2", 30)); }
    void addSides()    { meal.add(new Food("Chicken kebab", 110)); }
}
class BeefKebabMealBuilder extends SetMealBuilder {
    void addMainItem() { meal.add(new Food("Naan x2", 30)); }
    void addSides()    { meal.add(new Food("Beef kebab", 140)); }
}
class Waiter {                                                   // director
    SetMeal prepare(SetMealBuilder b, String drink, double drinkPrice) {
        b.addMainItem();
        b.addSides();
        if (drink != null) b.addDrink(drink, drinkPrice);       // drinks are optional extras
        return b.getMeal();
    }
}
```

```
public class LalbaghDemo {
    public static void main(String[] args) {
        Waiter w = new Waiter();
        System.out.println("Rice set + lemonade:");
        w.prepare(new RiceMealBuilder(), "Lemonade", 35).show();
        System.out.println("Bread + beef kebab, no drink:");
        w.prepare(new BeefKebabMealBuilder(), null, 0).show();
    }
}
```

(d) Three patterns, each fitting one requirement (prices assumed: Veggi 300, Beef 400, Chicken 350, cheese +50, Coffee 80, Water 20, Coke 40, French Fry 100, Onion Rings 120): **Decorator** — extra cheese wraps a pizza and adds to price(); **Composite** — an order is a MenuItem containing MenuItems, so price() of an order = sum of its parts; **Builder** — a combo meal is assembled step by step (pizza, appetizer, drinks) by ComboBuilder.

```
// 2020-21 Q1(a) — the orders built with three patterns: Decorator (cheese), Composite (orders),
Builder (combo meals)
import java.util.ArrayList;
import java.util.List;

interface MenuItem { String desc(); double price(); }          // common component interface
class ShopPizza implements MenuItem {
    private final String kind; private final double p;
    ShopPizza(String kind, double p) { this.kind = kind; this.p = p; }
    public String desc() { return kind + " pizza"; }
    public double price() { return p; }
}
class SimpleItem implements MenuItem {                         // drinks, appetizers
    private final String n; private final double p;
    SimpleItem(String n, double p) { this.n = n; this.p = p; }
    public String desc() { return n; }
    public double price() { return p; }
}
// assumed prices: Veggi 300, Beef 400, Chicken 350, cheese +50; Coffee 80, Water 20, Coke 40;
//                 French Fry 100, Onion Rings 120

// Pattern 1 — DECORATOR: extra cheese wraps any pizza
abstract class PizzaDecorator implements MenuItem {
    protected final MenuItem inner;
    PizzaDecorator(MenuItem i) { inner = i; }
}
class ExtraCheese extends PizzaDecorator {
    ExtraCheese(MenuItem i) { super(i); }
    public String desc() { return inner.desc() + " + cheese"; }
    public double price() { return inner.price() + 50; }
}

// Pattern 2 — COMPOSITE: an order is a MenuItem made of MenuItems
class Order implements MenuItem {
    private final String name;
    private final List<MenuItem> parts = new ArrayList<>();
    Order(String name) { this.name = name; }
    Order add(MenuItem m) { parts.add(m); return this; }
    public String desc() {
        List<String> d = new ArrayList<>();
        for (MenuItem m : parts) d.add(m.desc());
        return name + " [" + String.join(", ", d) + "]";
    }
    public double price() { double s = 0; for (MenuItem m : parts) s += m.price(); return s; }
}

// Pattern 3 — BUILDER: a combo meal is assembled step by step (appetizer, pizza, drinks)
```

```
class ComboMeal {
    final List<MenuItem> items = new ArrayList<>();
    double price() { double s = 0; for (MenuItem m : items) s += m.price(); return s; }
}
class ComboBuilder {
    private final ComboMeal meal = new ComboMeal();
    ComboBuilder pizza(String kind, double p, boolean cheese) {
        MenuItem pz = new ShopPizza(kind, p);
        meal.items.add(cheese ? new ExtraCheese(pz) : pz);
        return this;
    }
    ComboBuilder appetizer(String n, double p) { meal.items.add(new SimpleItem(n, p)); return this; }
    ComboBuilder drink(String n, double p)     { meal.items.add(new SimpleItem(n, p)); return this; }
    ComboMeal build() { return meal; }
}
public class BurgerShop {
    public static void main(String[] args) {
        MenuItem o1 = new Order("Order 1").add(new ExtraCheese(new ShopPizza("Beef", 400)))
                                          .add(new SimpleItem("French Fry", 100));
        MenuItem o2 = new Order("Order 2").add(new ShopPizza("Chicken", 350))
                                          .add(new SimpleItem("Onion Rings", 120)).add(new
SimpleItem("Water", 20));
        ComboMeal o3 = new ComboBuilder().pizza("Veggi", 300, false).appetizer("French Fry", 100)
                                         .drink("Coke", 40).drink("Coke", 40).build();
        ComboMeal o4 = new ComboBuilder().pizza("Veggi", 300, false).appetizer("Onion Rings", 120)
                                         .drink("Coffee", 80).drink("Water", 20).build();
        System.out.println(o1.desc() + " = " + o1.price());
        System.out.println(o2.desc() + " = " + o2.price());
        System.out.println("Order 3 (combo) = " + o3.price());
        System.out.println("Order 4 (combo) = " + o4.price());
    }
}
```

Output: Order 1 = 400 + 50 + 100 = 550; Order 2 = 350 + 120 + 20 = 490; Order 3 = 300 + 100 + 2 × 40 = 480; Order 4 = 300 + 120 + 80 + 20 = 520.

Pattern	Advantages	Disadvantages
Decorator	Add any number of extras at run time (double cheese = two wrappers); no subclass explosion (BeefWithCheese …); OCP	Many small objects; order of wrapping can matter; harder to debug
Composite	Orders and items are treated uniformly; nested combos; price() recursion is automatic	Hard to restrict what a composite may contain (e.g. 'exactly one pizza')
Builder	Step-by-step construction; fixed recipe for combos; validation before build()	Extra builder classes; overkill for single items

(e) Use the table in §4. Key contrasts: Factory Method vs Abstract Factory — one product via subclass override vs a consistent family via a factory object; Builder — one complex product in steps, versus factories that return a product in one call; Bridge — structural, not creational: it avoids an abstraction × implementation class explosion by composition (Notification has a MessageSender). Draw one tiny diagram per pattern.

Quick Notes — Creational Patterns

Revision points for Lessons 5–8.

L5 · Factory Method

- Intent: interface for creating an object, subclasses decide the class.
- Creator.factoryMethod() abstract; ConcreteCreator returns ConcreteProduct; client sees Product only.
- Cues: client must not call new; centralised creation; add types without editing.
- Slides: ShapeFactory, PaymentProcessor kiosk, electricity Plan (3.50/7.50/5.50), Logistics, Dialog.

L6 · Abstract Factory

- Families of related products, consistent; one concrete factory per family.
- Rows = families, columns = product kinds. Pizza (Italy/America), set menus, Uber/Pathao, GUI per OS.
- Liability: adding a new product kind changes every factory.

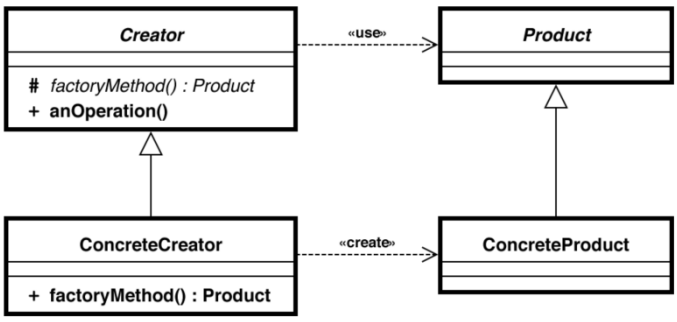
L7 · Singleton

- Private static instance + private constructor + public static getInstance().
- Eager = at class load (thread-safe); lazy = on first call (needs synchronized / DCL / holder).
- Variable number of instances: count in constructor (Derived1 ≤ 5, Derived2 ≤ 10).

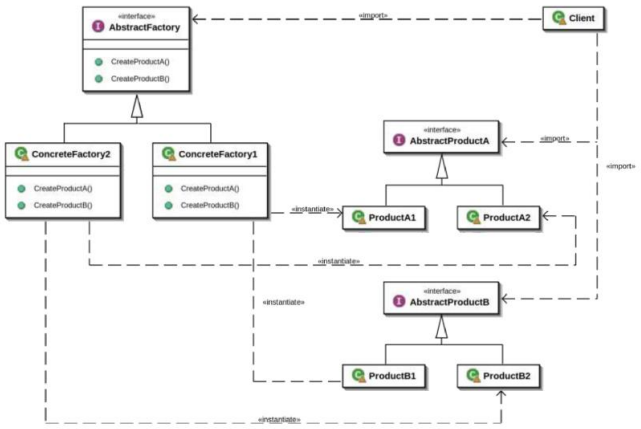
L8 · Builder

- Director fixes the steps; ConcreteBuilder makes one representation; getResult() returns the product.
- Cues: 'complex object', 'step by step', 'same process, different representations', director named.
- Burger shop: Decorator (cheese) + Composite (orders) + Builder (combos).

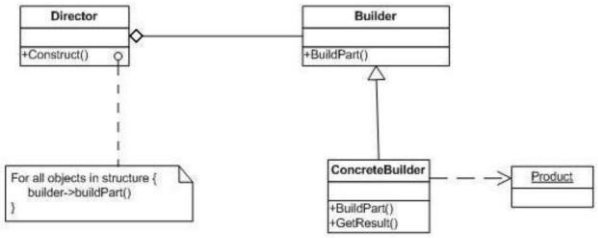
Key Diagrams to Reproduce



Factory Method (sl. 5)



Abstract Factory (sl. 18)



Builder (sl. 44)